Week 3 — SQL for Data Analysis

Industrial Training TR-103 · Data Analytics Organisation: World With Web **Trainee:** Savreet Kaur · URN 2302668 **Period:** 29 June 2026 – 3 July 2026

Objective of the week

Most real data lives in a database, not in a CSV file. This week covers querying it directly: filtering, aggregating, joining tables, and the window functions that make analytical SQL distinct from ordinary application SQL. The week ends by connecting the database back to pandas.

Day 11 — Monday, 29 June 2026

Relational database concepts

TERM	MEANING
Table	A collection of rows with the same columns
Row / record	One entity — a single order, customer or product
Column / field	One attribute, with a fixed data type
Primary key	A column that uniquely identifies each row
Foreign key	A column referencing another table's primary key
Schema	The structure: tables, columns, types and relationships

The practice database used throughout the week had four tables — `customers`, `orders`, `products` and `order_items` — linked by keys. Working against several related tables rather than one flat file is the point of the exercise, since joining is where SQL earns its place.

SELECT

```
SELECT customer_name, city, signup_date
FROM customers;

SELECT * FROM orders LIMIT 10;

SELECT DISTINCT city FROM customers;
```

Filtering with WHERE

```
SELECT * FROM orders
WHERE order_total > 5000
  AND order_date >= '2026-01-01';

SELECT * FROM customers
WHERE city IN ('Ludhiana', 'Amritsar', 'Jalandhar');

SELECT * FROM products
WHERE product_name LIKE '%Laptop%';

SELECT * FROM orders
WHERE shipped_date IS NULL;
```

Operators covered: comparison operators, `AND` / `OR` / `NOT`, `BETWEEN`, `IN`, `LIKE` with `%` and `_`, and `IS NULL`.

A point that was emphasised: `= NULL` never matches anything, because `NULL` means *unknown* and an unknown value is not equal to anything, including another unknown. `IS NULL` is required.

Sorting and limiting

```
SELECT customer_name, order_total
FROM orders
ORDER BY order_total DESC
LIMIT 10;
```

Takeaway

The logical order in which a query is evaluated — `FROM`, then `WHERE`, then `SELECT`, then `ORDER BY` — is not the order it is written in. Knowing this explains several errors encountered later in the week.

Day 12 — Tuesday, 30 June 2026

Aggregate functions

```
SELECT COUNT(*)          AS total_orders,
       SUM(order_total)  AS revenue,
       AVG(order_total)  AS avg_order,
       MIN(order_total)  AS smallest,
       MAX(order_total)  AS largest
FROM orders;
```

`COUNT(*)` counts rows; `COUNT(column)` counts non-null values in that column. The difference is a reliable way to count missing values.

GROUP BY

```
SELECT city,
       COUNT(*)         AS orders,
       SUM(order_total) AS revenue
FROM orders
GROUP BY city
ORDER BY revenue DESC;
```

This is the direct equivalent of pandas' `groupby` from Week 2, and the comparison was drawn explicitly.

HAVING versus WHERE

```
SELECT city, SUM(order_total) AS revenue
FROM orders
WHERE order_date >= '2026-01-01'   -- filters ROWS, before grouping
GROUP BY city
HAVING SUM(order_total) > 100000;  -- filters GROUPS, after grouping
```

CLAUSE	FILTERS	RUNS
`WHERE`	Individual rows	Before `GROUP BY`
`HAVING`	Grouped results	After `GROUP BY`

An aggregate function cannot appear in `WHERE`, because at that point the groups do not yet exist. This was the most common error of the day.

Takeaway

Any column in the `SELECT` list that is not inside an aggregate function must appear in `GROUP BY`. Some databases allow otherwise and return arbitrary values, which is worse than an error.

Day 13 — Wednesday, 1 July 2026

Joins

JOIN	RETURNS
`INNER JOIN`	Only rows matching in both tables
`LEFT JOIN`	All rows from the left table, matches where they exist
`RIGHT JOIN`	All rows from the right table
`FULL OUTER JOIN`	All rows from both, matched where possible

```
SELECT c.customer_name,
       o.order_id,
       o.order_total
FROM customers AS c
INNER JOIN orders AS o
        ON c.customer_id = o.customer_id;
```

LEFT JOIN as an analytical tool

```
-- Customers who have never ordered
SELECT c.customer_name
FROM customers AS c
LEFT JOIN orders AS o
       ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

This pattern — left join, then filter for nulls — answers "what is missing on one side", which an inner join cannot express.

Multi-table joins

```
SELECT c.customer_name,
       p.product_name,
       oi.quantity,
       oi.quantity * p.price AS line_total
FROM customers  AS c
JOIN orders     AS o  ON c.customer_id = o.customer_id
JOIN order_items AS oi ON o.order_id   = oi.order_id
JOIN products   AS p  ON oi.product_id = p.product_id;
```

A mistake worth recording

An early attempt produced far more rows than the orders table contained. The cause was a join condition left out, which produces a cross join — every row paired with every row. Checking the row count before and after a join was adopted as a standard check for the rest of the week.

Takeaway

An unexpected row count after a join is almost always a wrong or missing join key, and it inflates every aggregate computed afterwards.

Day 14 — Thursday, 2 July 2026

Subqueries

```
-- Orders above the overall average
SELECT order_id, order_total
FROM orders
WHERE order_total > (SELECT AVG(order_total) FROM orders);
```

Subqueries were covered in three positions: in `WHERE`, in `FROM` as a derived table, and in `SELECT` as a scalar value.

Common Table Expressions

```
WITH city_revenue AS (
    SELECT city, SUM(order_total) AS revenue
    FROM orders
    GROUP BY city
)
SELECT city, revenue
FROM city_revenue
WHERE revenue > 100000
ORDER BY revenue DESC;
```

A CTE and a subquery can express the same thing, but a CTE is named, appears at the top, and can be referenced more than once. Several CTEs can be chained, which turns a long nested query into a readable sequence of steps.

Practice

A nested query written on Day 13 was rewritten using CTEs. The result was the same and the query became legible, which was the point of the exercise.

Takeaway

Beyond one level of nesting, a CTE is almost always the better choice. SQL that cannot be read cannot be reviewed.

Day 15 — Friday, 3 July 2026

Window functions

A window function performs a calculation across a set of rows **without collapsing them**, which is the key difference from `GROUP BY`.

```
SELECT customer_name,
       order_total,
       ROW_NUMBER() OVER (ORDER BY order_total DESC) AS row_num,
       RANK()       OVER (ORDER BY order_total DESC) AS rank_pos,
       DENSE_RANK() OVER (ORDER BY order_total DESC) AS dense_pos
FROM orders;
```

FUNCTION	BEHAVIOUR ON TIES
`ROW_NUMBER()`	Always distinct; ties broken arbitrarily
`RANK()`	Same rank for ties, then skips numbers
`DENSE_RANK()`	Same rank for ties, no gaps

PARTITION BY

```
-- Top 3 orders within each city
SELECT * FROM (
    SELECT city,
           customer_name,
           order_total,
           ROW_NUMBER() OVER (PARTITION BY city
                              ORDER BY order_total DESC) AS rn
    FROM orders
) ranked
WHERE rn <= 3;
```

Running totals and period comparison

```
SELECT order_date,
       daily_total,
       SUM(daily_total) OVER (ORDER BY order_date)        AS running_total,
       LAG(daily_total, 1) OVER (ORDER BY order_date)     AS previous_day
FROM daily_sales;
```

`LAG` and `LEAD` make month-on-month comparison possible in a single query, without a self-join.

Connecting SQL to Python

```
import pandas as pd
import sqlite3

conn = sqlite3.connect("retail.db")
df = pd.read_sql_query("""
    SELECT city, SUM(order_total) AS revenue
    FROM orders
    GROUP BY city
""", conn)
conn.close()
```

The guidance given was to do the filtering and aggregation in SQL, close to the data, and bring back only the result set for visualisation in Python. Pulling a whole table into pandas and aggregating there wastes memory and time.

Takeaway

Window functions answer "rank within group" and "compare to previous period", which are two of the most common analytical requests and are awkward in pandas by comparison.

Summary of Week 3

DAY	DATE	TOPIC
11	Mon 29 Jun	RDBMS concepts, SELECT, WHERE, ORDER BY
12	Tue 30 Jun	Aggregates, GROUP BY, HAVING
13	Wed 1 Jul	Joins across multiple tables
14	Thu 2 Jul	Subqueries and CTEs
15	Fri 3 Jul	Window functions; SQL from Python

Skills gained: writing analytical SQL from scratch · joining several related tables correctly · expressing a multi-step analysis with CTEs · ranking and running totals with window functions · querying a database from pandas.

Next week: turning query results into charts and dashboards.